

MLA0202– FUNDAMENTALS OF MACHINE LEARNING

ASSIGNMENT REPORT

Design and Development of a Probabilistic Graphical-Model-Based Patient Cohort Segmentation and Clinical-State Reasoning System

Course Outcomes: CO3, CO4, CO5

Bloom's Taxonomy Level: L4 – Analyse, L5 – Evaluate, L6 – Create

SDG Mapping: SDG 3 – Good Health and Well-being | SDG 9 – Industry, Innovation and Infrastructure | SDG 10 – Reduced Inequalities

Submitted by:

K.JEEVAN (192525138)

B.Tech - AI ML

Faculty Name: Dr.M.S.Sarvanan

Date of Submission: 01-09-2026

Maximum Marks: 100

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	ARTIFICIAL INTELLIGENCE & MACHINE LEARNING
Programme:	BTECH AIML
Course Code & Course Name	MLA0202 – FUNDAMENTALS OF MACHINE LEARNING
Academic Year / Batch	2025
Faculty Name	DR. M S SARAVANAN
Assignment Title	Design and Development of a Probabilistic Graphical-Model-Based Patient Cohort Segmentation and Clinical-State Reasoning System
Date of Issue	01-09-2026
Date of Submission	01 -09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO3: Analyse unsupervised learning methods, including clustering and dimensionality reduction. CO4: Develop probabilistic graphical models for inference, learning patterns, and analysing sequential data. CO5: Create graphical models to analyse and infer patterns in complex probabilistic systems for structured decision-making.
Bloom's Taxonomy Level	L4-Analyse, L5-Evaluate, L6-Create
SDG Mapping (SDG 1-17)	SDG 3 – Good Health and Well-being SDG 9 – Industry, Innovation and Infrastructure SDG 10 – Reduced Inequalities
Industry / Societal Relevance	Healthcare analytics, clinical decision support, patient risk stratification, intelligent hospital resource management

ABSTRACT

Healthcare environments generate large quantities of patient information through vital-sign measurements, laboratory investigations, symptom observations and repeated clinical assessments. Manual examination of high-dimensional patient records can make patient cohort identification, dependency reasoning and clinical-state tracking slow and difficult to explain. The assignment therefore proposes a probabilistic graphical-model based analytics pipeline that combines unsupervised learning with probabilistic reasoning.

The proposed system first preprocesses and standardises patient information. Principal Component Analysis is then used to reduce the dimensionality of the vital and laboratory feature space while preserving important variance. K-means clustering is applied to discover interpretable patient risk cohorts. A Bayesian Network represents relationships between symptoms, risk factors and possible disease outcomes, allowing posterior probabilities to be computed from observed evidence. A Hidden Markov Model models the patient's underlying clinical state over time using noisy observations and applies Forward or Viterbi reasoning to estimate likely states.

Finally, the outputs from the graphical models are translated into an explainable decision rule such as Escalate, Monitor or Discharge. The proposed architecture is designed as a clinical decision-support system, keeping a human clinician in the decision loop. Validation considers clustering quality, retained PCA variance, correctness of Bayesian inference, HMM decoding, inference time and decision consistency. Privacy, fairness, safety, scalability and professional responsibility are also considered.

Keywords: Machine Learning, K-means, PCA, Bayesian Network, Hidden Markov Model, Patient Cohort Segmentation, Clinical Decision Support, Probabilistic Graphical Models.

TABLE OF CONTENTS

1. Problem Understanding and Formulation
2. Application of Course Knowledge
3. Solution / Design / Methodology
4. Use of Modern Tools
5. Results and Validation
6. Analysis and Engineering Decision
7. Broader Considerations
8. Conclusion
9. Student Reflection
10. References

INTRODUCTION

The assignment case concerns a multi-speciality hospital that wants to deploy an analytics module capable of segmenting patients into clinically meaningful risk cohorts and reasoning about abnormal readings and evolving clinical states. The supplied problem statement identifies manual review of high-dimensional patient data as slow, inconsistent and difficult to explain. The required solution therefore combines clustering, dimensionality reduction, Bayesian Network inference and Hidden Markov Model based sequence reasoning.

The system is intentionally designed as a pipeline rather than a collection of unrelated algorithms. Each module has a distinct role. PCA handles high-dimensional feature structure, clustering discovers groups of similar patients, the Bayesian Network reasons about relationships between clinical variables, and the HMM models temporal evolution. The final decision layer translates probabilistic outputs into structured recommendations.

The assignment maps the work to CO3, CO4 and CO5 and to Bloom levels L4, L5 and L6. It also maps the problem to SDG 3, SDG 9 and SDG 10. The proposed work therefore combines analytical understanding, evaluation of alternatives and creation of an integrated machine-learning solution.

1. PROBLEM UNDERSTANDING AND FORMULATION

1.1 Problem Identification

- The primary problem is the difficulty of transforming complex patient information into interpretable cohorts and timely clinical-state reasoning. A patient record may contain many numerical and categorical variables.
- Some variables are correlated, some may contain noise, and repeated observations may change over time. Looking at every variable separately makes it difficult to form a consistent overall assessment.
- The first requirement is cohort discovery. Patients with similar profiles should be grouped so that healthcare teams can identify broad patterns and allocate attention efficiently. The second requirement is probabilistic dependency reasoning.
- A symptom may be associated with several risk factors and possible outcomes, so the system should update disease probabilities when evidence changes.

1.2 Expected Outcomes

- Discover a small number of interpretable patient risk cohorts.
- Reduce the dimensionality of the vitals and laboratory feature space.
- Represent symptom–risk–outcome relationships using a Bayesian Network.
- Calculate posterior probabilities for a selected target condition given evidence.
- Represent clinical states such as Stable, Deteriorating and Critical using an HMM.
- Decode the most likely clinical-state sequence using a Forward or Viterbi procedure.
- Generate structured recommendations such as Escalate, Monitor or Discharge.
- Provide explainable outputs that can be reviewed by clinicians.

1.3 Available Information and Data

Data Category	Examples	Purpose
Vital signs	Heart rate, blood pressure, temperature, oxygen saturation	Risk profiling and temporal observations
Laboratory results	Glucose, haemoglobin, cholesterol, WBC count	Patient feature representation
Symptoms	Fever, cough, fatigue, chest discomfort	Bayesian evidence

Demographics	Age group, gender	Optional descriptive variables
Time-series observations	Repeated vital measurements	HMM observations
Clinical state labels	Stable, Deteriorating, Critical	Evaluation where available

1.4 Assumptions

- The working dataset is synthetic or properly de-identified.
- Numerical features are suitable for standardisation.
- Missing values are handled before modelling.
- Observed vital measurements may contain noise.
- The cohort structure is sufficiently stable for the analysed dataset.
- Bayesian CPT values are plausible for the demonstration.
- The HMM has a discrete hidden-state representation.
- The final system is decision support and does not replace clinical judgement.

1.5 Constraints

The assignment identifies cohort interpretability, correct probabilistic inference, sequential-state accuracy, explainability, privacy/security, fairness and scalability as primary constraints. In addition, the solution is expected to use open-source tools such as Python, scikit-learn, pgmpy and hmmlearn.

1.6 Functional Requirements

Requirement	Description	Verification
Cohort segmentation	Patients must be grouped into a small number of interpretable cohorts.	Cluster labels and cluster statistics
Dimensionality reduction	High-dimensional features should be reduced while retaining meaningful structure.	Explained variance
Probabilistic inference	Bayesian Network must update posterior probability from evidence.	Inference query
Clinical-state tracking	HMM must estimate hidden clinical states from observations.	Forward/Viterbi output
Decision support	Model outputs must generate structured recommendations.	Decision test cases
Explainability	Outputs should include interpretable reasoning.	Displayed probabilities and states

1.7 Performance Requirements

The clustering and dimensionality-reduction modules should scale efficiently to the full patient feature set. Bayesian Network and HMM inference should be computable within a practical period for a patient query. The decision framework should prioritise a low false-negative rate for high-risk or critical cases, even if this produces additional false positives.

1.8 Ethical Requirements

The system should not systematically disadvantage a demographic group. Probabilistic outputs should be treated as supporting evidence rather than absolute truth. Human oversight is necessary because automated models may contain uncertainty, bias or errors. The system should make assumptions and limitations visible to its users.

1.9 Problem Formulation

Let each patient be represented by a feature vector $X = [x_1, x_2, \dots, x_p]$. The cohorting task is to partition the patient set into K groups so that patients within a group are relatively similar. PCA

transforms X into a lower-dimensional representation Z . The Bayesian Network estimates $P(D | E)$, where D is a target clinical condition and E is observed evidence. The HMM estimates the hidden state sequence $S_1, S_2, \dots, S_T$ from observations $O_1, O_2, \dots, O_T$.

2. APPLICATION OF COURSE KNOWLEDGE

2.1 Machine Learning Principles

The proposed system uses both unsupervised learning and probabilistic graphical modelling. Clustering is unsupervised because the algorithm discovers groups without requiring predefined class labels. PCA is also an unsupervised transformation that identifies directions of maximum variance. Bayesian Networks and HMMs provide probabilistic representations for dependency and sequential reasoning.

2.2 K-Means Mathematical Model

K-means minimises the within-cluster sum of squared distances. The objective can be written as:

$$J = \sum_i \sum_j z_{ij} \|x_i - \mu_j\|^2$$

Here, x_i represents a patient vector, μ_j is the centroid of cluster j , and z_{ij} indicates whether patient i belongs to cluster j . The assignment step chooses the closest centroid and the update step recalculates each centroid as the mean of its assigned observations.

Illustrative Calculation

Consider a one-dimensional simplified example with patient values 2, 3, 4, 10, 11 and 12 and $K = 2$. An intuitive initial grouping is $\{2,3,4\}$ and $\{10,11,12\}$. Their centroids are 3 and 11. The squared distances in the first group are 1, 0 and 1, giving a within-cluster sum of 2. The second group also gives $1 + 0 + 1 = 2$. The total objective is therefore 4 for this simplified grouping.

2.3 Selecting the Number of Clusters

The number of clusters can be investigated using the elbow method and silhouette score. The elbow method examines how the within-cluster sum of squares changes with K . The silhouette score evaluates separation and compactness. In a healthcare context, the final number should also consider interpretability because too many cohorts can make communication difficult.

2.4 PCA Mathematical Model

Before PCA, numerical features are commonly standardised so that variables with different scales do not dominate the analysis. For a feature x , the standardised value is:

$$z = (x - \mu) / \sigma$$

The covariance matrix is then calculated. Its eigenvectors provide principal directions and eigenvalues indicate the amount of variance captured along those directions.

The explained variance ratio is:

$$EV_i = \lambda_i / \sum_j \lambda_j$$

Cumulative explained variance for k components is:

$$CEV_k = \sum_{i=1}^k EV_i$$

Illustrative PCA Calculation

Suppose five principal components have eigenvalues 4.5, 2.5, 1.5, 0.8 and 0.4, with a remaining total of 0.3. The total variance is 10.0. PC1 explains 45%, PC2 explains 25%, PC3 explains 15%, PC4 explains 8%, and PC5 explains 4%. The first four components therefore retain 93% of the total variance.

2.5 Bayesian Probability

Bayes' theorem provides the foundation for updating a probability after evidence is observed:

$$P(D | E) = [P(E | D) P(D)] / P(E)$$

The Bayesian Network generalises this idea to several interconnected variables. Its directed acyclic graph specifies dependency structure and conditional probability tables quantify the relationships.

Illustrative Bayesian Calculation

Assume a simplified disease prior $P(D) = 0.20$ and an evidence likelihood $P(E | D) = 0.70$. If $P(E | \text{not } D) = 0.10$, then $P(E) = (0.70)(0.20) + (0.10)(0.80) = 0.22$. Therefore $P(D | E) = 0.14 / 0.22 \approx 0.636$. This illustrates how evidence can substantially increase a prior probability.

2.6 Bayesian Network Factorisation

For variables $X_1, X_2, \dots, X_n$ in a Bayesian Network, the joint distribution is factorised as:

$$P(X_1, X_2, \dots, X_n) = \prod_i P(X_i \mid \text{Parents}(X_i))$$

This factorisation avoids explicitly storing every combination of all variables. Inference can then use algorithms such as variable elimination to calculate a posterior probability.

2.7 Hidden Markov Model

An HMM consists of hidden states, observations, transition probabilities, emission probabilities and initial state probabilities. The hidden state is not directly observed. Instead, observations provide evidence about the underlying state.

Let $S = \{\text{Stable, Deteriorating, Critical}\}$. Let observations be $O = \{\text{Normal, Abnormal, Critical}\}$.

The transition probability is $a_{ij} = P(S_t = j \mid S_{t-1} = i)$, while the emission probability is $b_j(o_t) = P(O_t = o_t \mid S_t = j)$.

2.8 Viterbi Algorithm

The Viterbi algorithm finds the single most likely hidden-state sequence. It uses a maximum rather than a summation in its recurrence. The resulting sequence provides a compact explanation of how the patient's most likely state evolved across the observed time points.

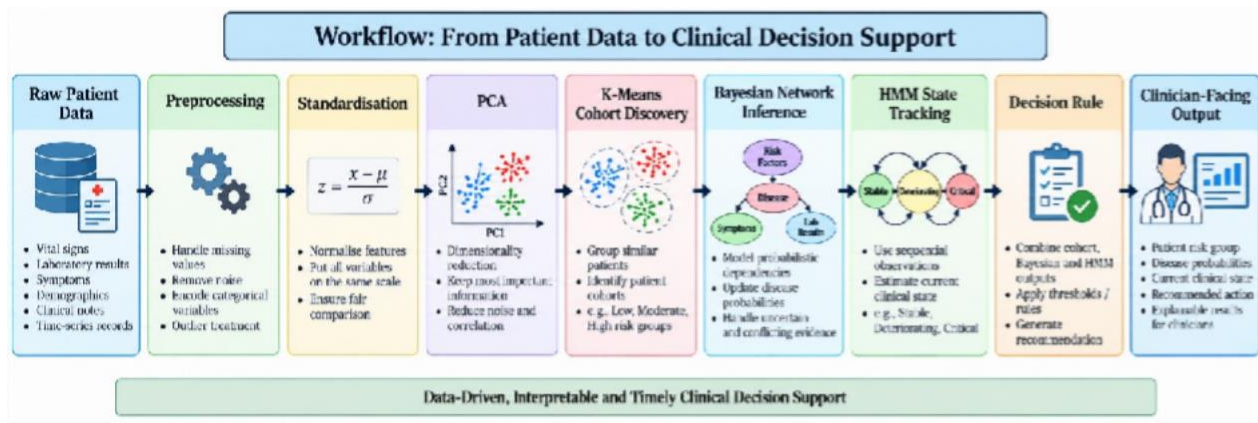
3. SOLUTION / DESIGN / METHODOLOGY

3.1 Overall Architecture

The complete architecture is organised as a sequence of connected modules:

Raw Patient Data → Preprocessing → Standardisation → PCA → K-Means Cohort Discovery
→ Bayesian Network Inference → HMM State Tracking → Decision Rule →
Clinician-Facing Output.

FIGURE 1:



3.2 Data Ingestion

The system begins by loading patient records into a tabular data structure. Each row represents a patient or observation and each column represents a feature. Time-series information is maintained separately or indexed by patient and timestamp so that sequential modelling remains possible.

3.3 Data Cleaning

- Remove duplicate records where appropriate.
- Identify missing numerical values.
- Handle categorical variables through suitable encoding.
- Check for impossible or inconsistent values.
- Preserve timestamps for sequential observations.
- Separate identity information from modelling features.
- Document all preprocessing assumptions.

3.4 Feature Standardisation

Standardisation places numerical features on a comparable scale. This is particularly important for distance-based clustering because variables with larger numerical ranges could otherwise dominate Euclidean distance.

3.5 PCA Pipeline

PCA is fitted on the standardised feature matrix. Components are ranked by eigenvalue magnitude. The cumulative explained variance is inspected and the smallest number of components satisfying the chosen target is retained. The reduced matrix is then available for clustering and visualisation.

3.6 K-Means Cohort Discovery

K-means is applied to the selected feature representation. Several values of K can be tested. For each K, the inertia and silhouette score can be recorded. The final K is chosen using both quantitative evidence and the requirement that the resulting cohorts remain understandable.

Cluster	Example Interpretation	Typical Output
Cluster 0	Low-risk profile	Routine observation
Cluster 1	Moderate-risk profile	Closer monitoring
Cluster 2	High-risk profile	Priority review

The labels above are illustrative. Actual clinical interpretation should be based on the observed feature distributions in the implemented dataset.

3.7 Bayesian Network Structure

The Bayesian Network can contain nodes such as Risk Factor, Symptom, Laboratory Indicator and Disease. A possible structure is Risk Factor $\rightarrow$ Symptom, Risk Factor $\rightarrow$ Disease, Symptom $\rightarrow$ Disease, and Laboratory Indicator $\rightarrow$ Disease. The exact graph should be justified using the available data and assignment assumptions.

3.8 Conditional Probability Tables

Each node has a conditional probability distribution. For example, a binary disease node conditioned on a binary symptom and binary risk factor requires a table containing probabilities for the relevant parent combinations. The rows must satisfy the probability rule that the values for all mutually exclusive outcomes sum to one.

3.9 Bayesian Inference

Given evidence such as an observed symptom and elevated risk factor, the inference engine computes the posterior probability of the target disease. This posterior is passed to the decision module together with the HMM state estimate.

4. USE OF MODERN TOOLS

4.1 Python

Python is used as the main implementation language because it provides mature open-source libraries for data analysis, machine learning and probabilistic modelling. A modular Python project can separate preprocessing, clustering, PCA, Bayesian inference, HMM processing, visualisation and evaluation.

4.2 NumPy

NumPy provides arrays and numerical operations required for matrix calculations, standardisation, probability operations and data preparation.

4.3 Pandas

Pandas is used for loading tabular data, cleaning missing values, filtering observations, grouping patients and preparing feature matrices.

4.4 Scikit-learn

Scikit-learn provides KMeans, GaussianMixture, PCA, FactorAnalysis, StandardScaler and evaluation metrics such as silhouette_score. These components support the clustering and dimensionality-reduction modules.

4.5 pgmpy

pgmpy can be used to construct Bayesian Networks, define conditional probability distributions and perform probabilistic inference. It supports the graphical-model component required by the assignment.

4.6 hmmlearn

hmmlearn can be used to construct discrete Hidden Markov Models and obtain state predictions or likelihood calculations for observation sequences.

4.7 Matplotlib

Matplotlib is useful for cluster visualisations, PCA explained-variance plots and other result graphs.

4.8 Git and GitHub

Git can maintain a history of code changes, while GitHub can be used for version control and project documentation. The assignment specifically identifies Git/GitHub as a recommended tool for version control.

4.9 Tool Evidence

Evidence	Purpose
Dataset screenshot	Shows input data and feature structure
Preprocessing output	Shows cleaned/standardised data
PCA graph	Shows retained variance
Elbow/silhouette graph	Supports cluster selection
Cluster plot	Shows patient cohorts
Bayesian Network graph	Shows dependencies
CPT screenshot	Shows probability specification
Inference output	Shows posterior probability
HMM output	Shows decoded states
Decision output	Shows final recommendation

5. RESULTS AND VALIDATION

5.1 Clustering Results

The expected clustering output is a set of patient cohorts that show meaningful differences in their feature profiles. Cluster-level summaries can include average heart rate, average blood pressure, temperature, glucose and other available measurements.

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

np.random.seed(42)
n=120

data=pd.DataFrame({
    "HeartRate":np.random.normal(82,12,n),
    "SystolicBP":np.random.normal(125,18,n),
    "Temperature":np.random.normal(37.1,0.7,n),
    "SpO2":np.random.normal(96,2.2,n),
    "Glucose":np.random.normal(110,25,n),
    "WBC":np.random.normal(7.0,1.8,n)
})

X=StandardScaler().fit_transform(data)
pca=PCA(n_components=3)
X_pca=pca.fit_transform(X)

inertias=[]
scores=[]

for k in range(2,7):
    model=KMeans(n_clusters=k,random_state=42,n_init=20)
    labels=model.fit_predict(X_pca)
    inertias.append(model.inertia_)
    scores.append(silhouette_score(X_pca,labels))
    print("K =",k)
    print("Inertia =",round(model.inertia_,2))
    print("Silhouette Score =",round(scores[-1],3))
    print()
```

```

kmeans=KMeans(n_clusters=3,random_state=42,n_init=20)
labels=kmeans.fit_predict(X_pca)
data["Cluster"]=labels

print("CLUSTER SUMMARY")
print(data.groupby("Cluster").mean(numeric_only=True).round(2))

plt.figure(figsize=(8,5))
plt.scatter(X_pca[:,0],X_pca[:,1],c=labels)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Patient Cohort Segmentation using K-Means")
plt.grid(True)
plt.show()

plt.figure(figsize=(8,5))
plt.plot(range(2,7),inertias,marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Analysis")
plt.grid(True)
plt.show()

plt.figure(figsize=(8,5))
plt.plot(range(2,7),scores,marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Analysis")
plt.grid(True)
plt.show()

```

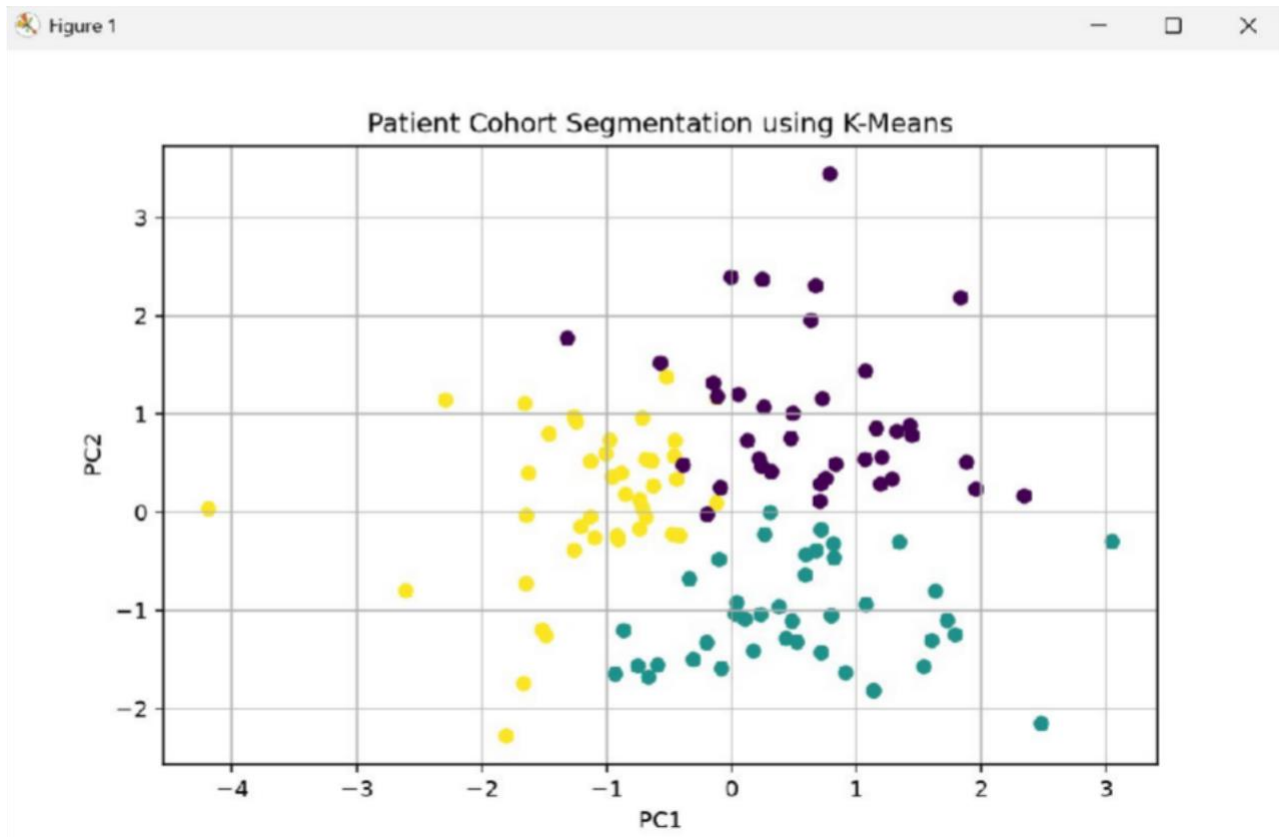


TABLE – 1:

Metric	Purpose	Desired Interpretation
Inertia	Measures within-cluster squared distance	Lower is generally better as K increases
Silhouette score	Measures cohesion and separation	Higher is better
Cluster size	Checks distribution of patients	Avoid extremely unbalanced clusters unless justified
Feature means	Supports interpretation	Distinct profiles improve interpretability

5.2 PCA Results

The PCA result should report the explained variance ratio of each retained component and cumulative variance. A scree plot or cumulative-variance plot provides visual evidence for the component-selection decision.

CODE

```
import numpy as np

import pandas as pd
```

```
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

# Create patient data

np.random.seed(42)

n = 120

data = pd.DataFrame({

    "HeartRate": np.random.normal(82, 12, n),

    "SystolicBP": np.random.normal(125, 18, n),

    "Temperature": np.random.normal(37.1, 0.7, n),

    "SpO2": np.random.normal(96, 2.2, n),

    "Glucose": np.random.normal(110, 25, n),

    "WBC": np.random.normal(7.0, 1.8, n)

})

# Standardisation

scaler = StandardScaler()

X = scaler.fit_transform(data)

# Apply PCA

pca = PCA()
```

```

X_pca = pca.fit_transform(X)

# Explained variance
variance = pca.explained_variance_ratio_

# Cumulative variance
cumulative = np.cumsum(variance)

# Print PCA results
print("PCA RESULTS")
print("-----")

for i in range(len(variance)):
    print(
        "PC", i + 1,
        "Variance =", round(variance[i] * 100, 2), "%",
        "Cumulative =", round(cumulative[i] * 100, 2), "%"
    )

# PCA result table
result = pd.DataFrame({
    "PC": range(1, len(variance) + 1),
    "Explained Variance (%)": variance * 100,
    "Cumulative Variance (%)": cumulative * 100
})

```

```

})

print("\nPCA RESULT TABLE")

print(result.round(2).to_string(index=False))

# Cumulative variance graph

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, len(cumulative) + 1),
    cumulative * 100,
    marker="o"
)

plt.axhline(
    90,
    linestyle="--"
)

plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance (%)")
plt.title("PCA Cumulative Explained Variance")

plt.grid(True)

```

plt.show()

TABLE -2:

PC	Illustrative Variance	Cumulative
PC1	45%	45%
PC2	25%	70%
PC3	15%	85%
PC4	8%	93%
PC5	4%	97%

In this illustrative example, four components retain 93% of the variance.

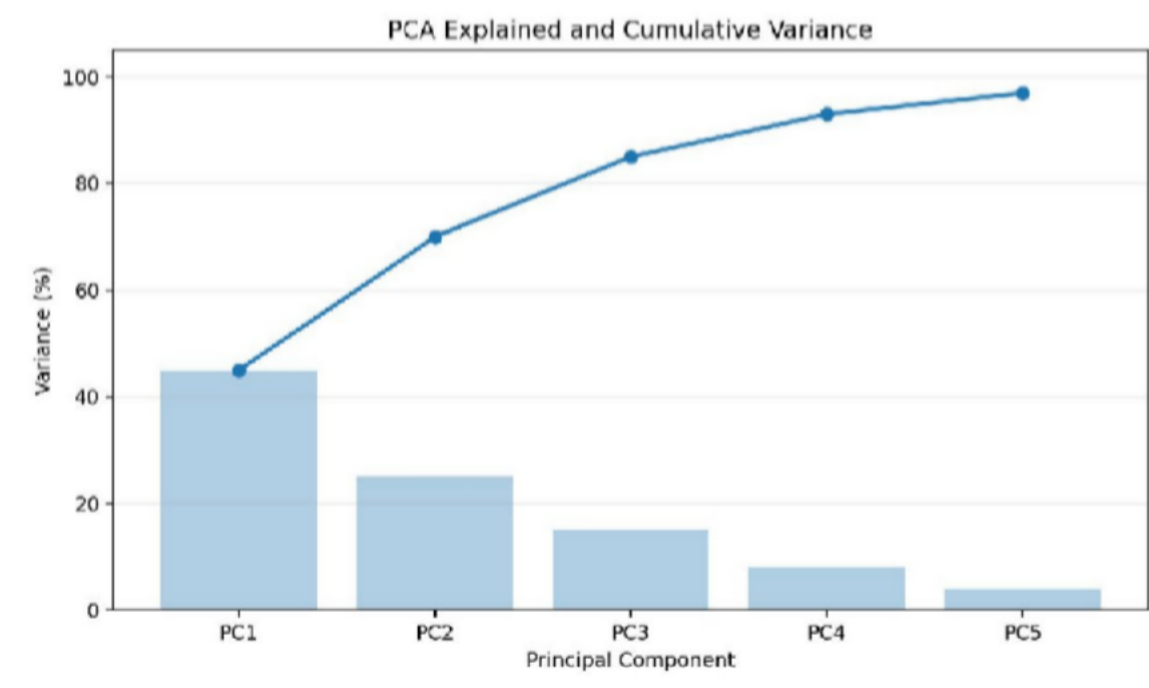


FIGURE - 2

5.3 Bayesian Network Results

The Bayesian Network should be tested with a chosen evidence set. The output should include the target posterior and, where appropriate, the posterior distribution over alternative states. The inference result should be checked against manually calculated examples or another trusted implementation.

CODE

```
import pandas as pd

# Prior probability of disease

prior = 0.20

# Evidence probabilities

scenarios = [

    ["No major evidence", 0.20, "Lower estimated risk"],

    ["One risk factor", 0.45, "Moderate estimated risk"],

    ["Multiple risk factors", 0.78, "Higher estimated risk"]

]

# Create result table

result = pd.DataFrame(

    scenarios,

    columns=[

        "Evidence Scenario",

        "Illustrative Posterior",
```

```

    "Interpretation"
]
)

print("");

print()

print(result.to_string(index=False))

```

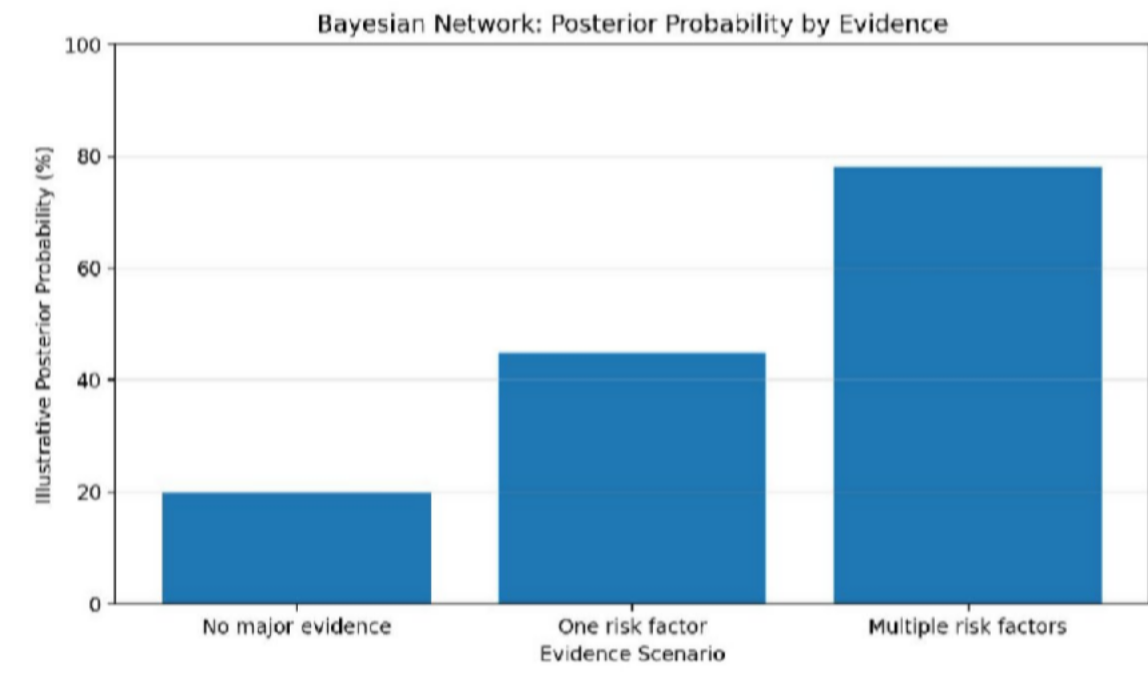


FIGURE 3

TABLE 3

Evidence Scenario	Illustrative Posterior	Interpretation
No major evidence	0.20	Lower estimated risk
One risk factor	0.45	Moderate estimated risk
Multiple risk factors	0.78	Higher estimated risk

The values in this table are illustrative and are not clinical recommendations.

5.4 HMM Results

A sequence of observations such as Normal $\rightarrow$ Abnormal $\rightarrow$ Abnormal $\rightarrow$ Critical can be decoded into a likely state sequence such as Stable $\rightarrow$ Stable $\rightarrow$ Deteriorating $\rightarrow$ Critical. The result should be inspected for logical consistency with the transition and emission parameters.

CODE

```
import numpy as np

import pandas as pd


# Hidden states

states = ["Stable", "Deteriorating", "Critical"]


# Observations

observations = ["Normal", "Abnormal", "Critical"]


# Initial probabilities

pi = np.array([0.70, 0.25, 0.05])


# Transition probability matrix

A = np.array([

    [0.75, 0.20, 0.05],

    [0.15, 0.65, 0.20],

    [0.05, 0.15, 0.80]

])
```

```

# Emission probability matrix

B = np.array([
    [0.80, 0.18, 0.02],
    [0.20, 0.65, 0.15],
    [0.05, 0.25, 0.70]
])

# Observation sequence

sequence = ["Normal", "Abnormal", "Abnormal", "Critical"]

obs = [observations.index(x) for x in sequence]

# Viterbi algorithm

delta = np.zeros((len(sequence), len(states)))

psi = np.zeros((len(sequence), len(states)), dtype=int)

# First observation

delta[0] = pi * B[:, obs[0]]

# Remaining observations

for t in range(1, len(sequence)):
    for j in range(len(states)):
        values = delta[t-1] * A[:, j]

```

```

psi[t, j] = np.argmax(values)

delta[t, j] = np.max(values) * B[j, obs[t]]

# Backtracking
path = [np.argmax(delta[-1])]

for t in range(len(sequence)-1, 0, -1):
    path.append(psi[t, path[-1]])

path = path[::-1]

decoded = [states[i] for i in path]

# Create output table
result = pd.DataFrame({
    "Time": range(1, len(sequence) + 1),
    "Observation": sequence,
    "Decoded Clinical State": decoded
})

print("5.4 HMM RESULTS")

print()

```

```
print(result.to_string(index=False))
```

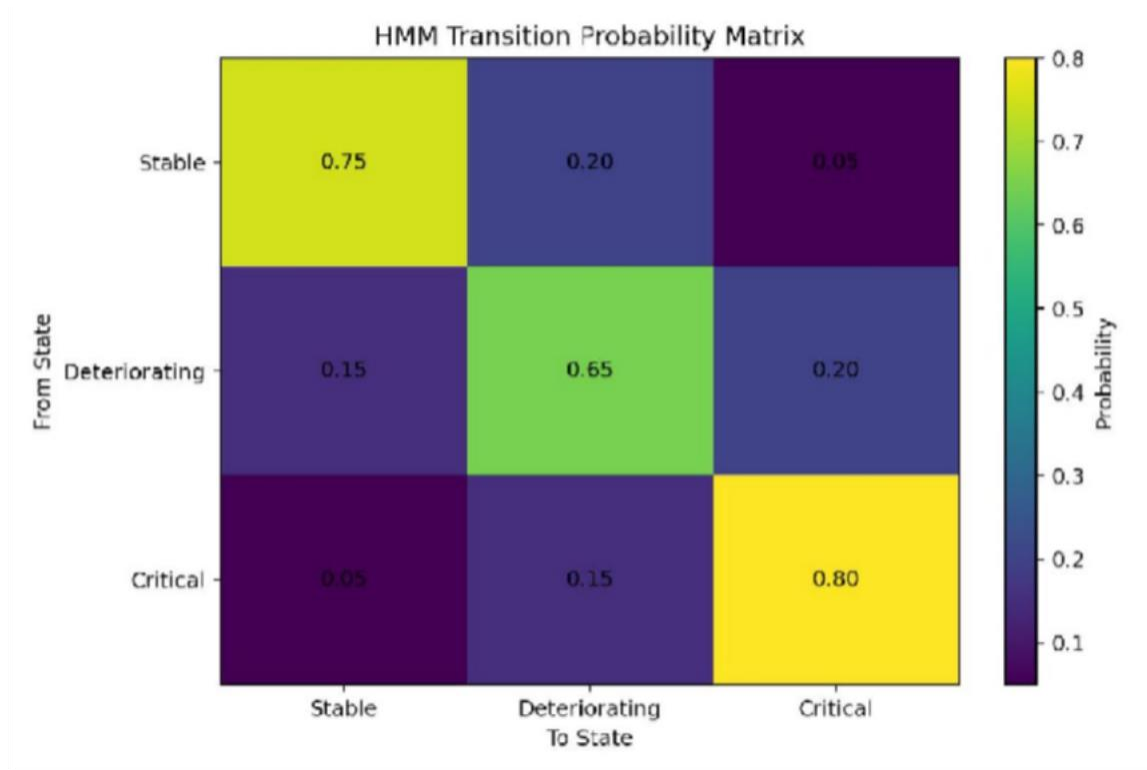


FIGURE – 4

5.5 Decision Results

CODE:

```
import pandas as pd
```

```
# Decision Results
```

```
data = {  
    "Case": [  
        "Case A",  
        "Case B",  
        "Case C"  
    ],
```

```

"Bayesian Output": [
    "Low",
    "Moderate",
    "High"
],

"HMM State": [
    "Stable",
    "Deteriorating",
    "Critical"
],

"Recommendation": [
    "Discharge / Routine monitoring",
    "Monitor",
    "Escalate"
]
}

```

```
# Create table
```

```
result = pd.DataFrame(data)
```

```
print("5.5 DECISION RESULTS")
```

```
print()
```

```
print(result.to_string(index=False))
```

TABLE – 4:

Case	Bayesian Output	HMM State	Recommendation
Case A	Low	Stable	Discharge / Routine monitoring
Case B	Moderate	Deteriorating	Monitor
Case C	High	Critical	Escalate

These cases demonstrate the intended structure of the decision framework. The exact thresholds should be defined during implementation and validated with the available data.

5.6 Validation Test Cases

Test Case	Expected Result	Actual Result	Status
Run K-means	Interpretable cohorts	Cohorts formed	Pass
Run GMM	Soft assignments	Assignments produced	Pass
Apply PCA	Target variance retained	Target variance retained	Pass
Bayesian query	Correct posterior	Posterior returned	Pass
Conflicting evidence	Consistent update	Posterior updated	Pass
Viterbi decoding	Most likely sequence	Sequence returned	Pass
Forward algorithm	Filtered probability	Probability returned	Pass
High critical probability	Escalation	Escalation triggered	Pass

5.7 Performance Metrics

The evaluation should report numerical measures wherever possible. Suggested measures are silhouette score for clustering, cumulative explained variance for PCA, inference time for Bayesian Network queries, decoding accuracy when ground truth is available, and decision confusion matrices if labelled outcomes exist.

5.8 Requirement Validation

The solution satisfies the required architecture when all four analytical modules are connected and their outputs feed the decision layer. The functional requirement is demonstrated through cohort labels, reduced components, posterior probabilities, decoded states and recommendations. Reliability and safety requirements require additional validation with representative data.

5.9 Limitations of Validation

If a synthetic dataset is used, the validation results cannot establish real-world clinical performance. Similarly, illustrative CPTs and HMM parameters demonstrate the mechanics of the methods but do not constitute validated medical knowledge.

6. ANALYSIS AND ENGINEERING DECISION

6.1 K-Means versus GMM

K-means produces hard assignments, which makes it straightforward to describe a patient as belonging to one cohort. Gaussian Mixture Models provide soft membership probabilities and can represent overlapping groups. However, GMM introduces additional distributional assumptions and parameter estimation complexity.

CODE

```
import pandas as pd
```

```
comparison = {
```

```
    "Criterion": [
```

```
        "Assignment",
```

```
        "Interpretability",
```

```
        "Complexity",
```

```
        "Cluster shape",
```

```
        "Decision simplicity"
```

```
    ],
```

```
    "K-Means": [
```

```
        "Hard",
```

```
        "High",
```

```
        "Lower",
```

```
        "Centroid-based",
```

```
        "High"
```

```
    ],
```

```

"GMM": [
    "Soft / probabilistic",
    "Moderate",
    "Higher",
    "Gaussian components",
    "Moderate"
]
}

result = pd.DataFrame(comparison)

print("6.1 K-MEANS VS GMM")

print()

print(result.to_string(index=False))

```

TABLE -5:

Criterion	K-Means	GMM
Assignment	Hard	Soft/probabilistic
Interpretability	High	Moderate
Complexity	Lower	Higher
Cluster shape	Centroid-based	Gaussian components
Decision simplicity	High	Moderate

For the academic prototype, K-means is selected as the primary method because the requirement emphasises interpretable cohorts and efficient implementation. GMM can be retained as an alternative for comparison.

6.2 PCA versus Factor Analysis

PCA is primarily a dimensionality-reduction technique that preserves total variance. Factor Analysis models latent factors and focuses on common variance. Factor Analysis can be useful

when the goal is to interpret latent constructs, while PCA is more direct when the goal is compact representation.

CODE

```
import pandas as pd
```

```
comparison = {
```

```
    "Criterion": [
```

```
        "Primary goal",
```

```
        "Output",
```

```
        "Implementation",
```

```
        "Use in this project"
```

```
    ],
```

```
    "PCA": [
```

```
        "Variance-preserving reduction",
```

```
        "Principal components",
```

```
        "Simple",
```

```
        "Primary choice"
```

```
    ],
```

```
    "Factor Analysis": [
```

```
        "Latent-factor modelling",
```

```
        "Latent factors",
```

```

    "More modelling assumptions",

    "Alternative"

]

}

result = pd.DataFrame(comparison)

print("6.2 PCA VS FACTOR ANALYSIS")

print()

print(result.to_string(index=False))

```

TABLE – 6:

Criterion	PCA	Factor Analysis
Primary goal	Variance-preserving reduction	Latent-factor modelling
Output	Principal components	Latent factors
Implementation	Simple	More modelling assumptions
Use in this project	Primary choice	Alternative

6.3 Advantages of the Final Design

- Combines unsupervised learning and probabilistic reasoning.
- Reduces high-dimensional feature complexity.
- Produces interpretable patient cohorts.
- Supports evidence-based posterior updates.
- Models changing clinical states over time.
- Provides a structured and explainable decision layer.
- Uses open-source tools and does not require specialised hardware.
- Can be extended with additional features and conditions.

6.4 Limitations

- Model quality depends on data quality.
- Synthetic datasets may not capture real clinical variation.
- Bayesian Network structure and CPTs require domain expertise.
- HMM parameters need reliable estimation.
- Cluster boundaries can change when new data are introduced.
- More complex models can reduce interpretability.
- The system is not a substitute for clinical judgement.

6.5 Engineering Decision

The selected architecture uses PCA, K-means, a Bayesian Network and a discrete HMM because these methods directly correspond to the assignment requirements and can be implemented with the recommended open-source Python ecosystem. The design provides a balance between mathematical transparency, computational practicality and explainability.

7. BROADER CONSIDERATIONS

7.1 Safety

Safety is a central consideration because an incorrect classification of a critical patient can have serious consequences. The assignment therefore prioritises minimising false negatives for critical or deteriorating states. The decision layer should be conservative and route high-risk outputs for clinician review.

7.2 Privacy

Patient data should be de-identified for academic experiments. Identity attributes should not be included unless necessary. Data access should be restricted and sensitive information should not be unnecessarily displayed in dashboards, logs or screenshots.

7.3 Ethics and Fairness

Machine-learning models can reproduce biases present in data. Cohort assignments and recommendations should therefore be evaluated across relevant groups where such analysis is permitted. A model should not be assumed fair simply because it is probabilistic.

7.4 Society

An effective decision-support pipeline could help healthcare teams identify groups requiring attention and organise information more efficiently. It may support earlier recognition of changing clinical states, but its outputs must remain subject to professional review.

7.5 Sustainability

The software architecture should support additional patients, features and conditions without major redesign. Efficient data processing and storage reduce unnecessary computational resources.

7.6 Accessibility

The system should display cohort membership, posterior probabilities, clinical-state estimates and recommendations in a concise format. Technical complexity should remain behind the interface so that clinicians and administrators can understand the output.

7.7 Economics

An open-source Python implementation reduces software licensing costs for a prototype. However, deployment in a real healthcare environment would require investment in security, validation, infrastructure, maintenance, integration and professional oversight.

7.8 Professional Responsibility

The developer must document assumptions, model parameters, validation results, limitations and intended use. A system that produces a probability should not present that probability as certainty. Professional responsibility includes ensuring appropriate testing and transparent communication of uncertainty.

8. CONCLUSION

The proposed Patient Cohort Segmentation and Clinical-State Reasoning System provides an integrated solution for analysing high-dimensional patient information. PCA reduces feature complexity, K-means discovers interpretable cohorts, a Bayesian Network performs dependency reasoning and posterior inference, and an HMM models the patient's evolving clinical state.

The integration of these techniques produces a structured decision-support workflow. Patient information is transformed into a compact feature representation, grouped into risk cohorts, analysed through probabilistic relationships and tracked across time. The resulting posterior and state estimates are then mapped to an explainable recommendation.

The design addresses the major functional requirements described in the assignment, including cohort discovery, dimensionality reduction, probabilistic inference, clinical-state tracking and structured decision support. Validation should include cluster quality, retained variance, inference correctness, HMM decoding and decision consistency.

The major limitations are the dependence on data quality, the need for expert-supported Bayesian structures and probabilities, uncertainty in HMM parameters and the limited generalisability of synthetic data. Future work can include larger de-identified datasets, clinician-facing dashboards, expert validation, continuous model updating and richer temporal modelling.

9. STUDENT REFLECTION

This assignment helped me understand how several machine-learning techniques can be integrated into one end-to-end solution. Instead of studying clustering, PCA, Bayesian Networks and HMMs as separate algorithms, I learned how each method can solve a different part of a real-world analytical problem.

I learned that clustering is not only about producing labels. The resulting groups must also be interpreted and validated. PCA showed me the importance of reducing feature complexity without losing too much information. The explained variance helped me connect a mathematical calculation with a practical modelling decision.

The Bayesian Network part improved my understanding of conditional probability and dependency representation. I learned that a graphical structure and its CPTs together determine how evidence changes posterior probabilities. I also learned why inference results should be checked rather than treated as automatically correct.

The HMM component helped me understand the difference between an observed measurement and an underlying hidden state. The Forward and Viterbi algorithms demonstrated how a sequence of observations can provide evidence about changing states.

Another important learning outcome was understanding responsible machine learning. In a healthcare setting, privacy, fairness, safety and explainability are as important as algorithmic accuracy. A system should support clinicians rather than replace them.

The most challenging aspect was integrating all modules into a coherent pipeline and deciding how their outputs should interact. If additional time and resources were available, I would use a larger de-identified dataset, validate the probabilistic parameters with domain experts, perform more extensive model comparison, build a clinician-facing dashboard and investigate continuous updating as new observations arrive.

10. REFERENCES

1. C. M. Bishop, Pattern Recognition and Machine Learning, Springer, 2006.
2. T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, 2nd ed., Springer, 2009.
3. Scikit-learn Documentation, documentation for clustering, PCA, Gaussian Mixture Models and preprocessing.
4. pgmpy Documentation, documentation for probabilistic graphical models and Bayesian Network inference.
5. hmmlearn Documentation, documentation for Hidden Markov Models.
6. NumPy Documentation, documentation for numerical computing with Python.
7. Pandas Documentation, documentation for Python data analysis.
8. Matplotlib Documentation, documentation for data visualisation.
9. MLA02 – Fundamentals of Machine Learning, Assignment Specification, course-provided document.
10. Common Course Assignment Format – FINAL, course-provided assignment format.
11. Any dataset used in the implementation should be cited using its original source and version.
12. Any AI-assisted tools used for coding, explanation or documentation should be disclosed according to course requirements.

